

🎯 CORE PRINCIPLE

Do not build ML + CV + NLP + RAG + Agent just to show technologies. Start with a real industry problem and a useful idea. Choose the minimum AI components needed to solve it well. Integrate them into one useful application. Evaluate the system. Deploy it. Explain its value, limitations and risks.

🚀 **Build an AI solution – not a collection of disconnected demos.**

🏁 FINAL AI PROJECT MINDSET

🎯 Not: "I used every AI technology."

✅ Better: "I solved a real problem using the right AI technologies."

🧠 HOW TO USE LLMs

LLMs can help with brainstorming, explaining concepts, reviewing code, suggesting analysis ideas, debugging and documentation.

- 🚫 **Do not ask an LLM to generate your entire project dataset & code.**
- 🧠 Understand every important piece of code you submit.
- 💬 Be able to explain your data, features, model, metrics and deployment choices.

	Industry
	Domain
	Idea / Opportunity
	Data Sources
	Data Collection
	Data Validation
	Data Understanding
	Problem Definition
	AI Approach Selection
	Preparation
	AI Component Development
	Evaluation
	Tuning / Improvement
	Application Integration
	Deployment
	Business Impact
	Monitoring

1. INDUSTRY & DOMAIN

Industry = broad business area such as Healthcare, Banking, Retail, Manufacturing, Education or Telecom.

Domain = a specific area inside the industry, such as Patient Care, Fraud, Sales, Quality Control or Customer Service.

FOCUS

Understand the domain before choosing an AI technology.

2. IDEA / OPPORTUNITY

Identify a useful idea in the chosen domain. Ask what decision, automation, prediction, classification, search, assistance or workflow improvement could be created.

-  Define the intended user and use case.
-  Identify expected input and output.
-  Explain why the idea is useful.
-  Check whether AI is actually needed.
-  Do not add GenAI or Agents only because they are popular.

IMPORTANT

An idea is a hypothesis. Validate it against data availability, technical feasibility and business value.

3. PROBLEM DEFINITION

Convert the idea into a precise technical problem. Define input, expected output, success criteria and constraints.

Example:

 Idea

Help a hospital identify high-risk patients and assist staff with relevant documents.

 Problems

Predict risk + classify relevant text/images + retrieve trusted documents + assist workflow.

 Components

DL-CV / NLP / RAG / Agent – only where justified.

 Success

Good technical metrics + useful application behavior + business value.

4. DATA & KNOWLEDGE SOURCES

-  Images/video – image classification or object detection.
-  Text – NLP text classification.
-  Documents – RAG knowledge base.
-  APIs/databases – live or application data.
-  Public/official sources – real-world domain data.

DATA RULE

Do not default to synthetic data or blindly search Kaggle. Investigate legitimate real-world sources first. If data is unavailable, document the limitation and justify alternatives.

✓ 5. DATA / KNOWLEDGE VALIDATION

- Are sources trustworthy and relevant?
- Is data representative?
- Are labels reliable?
- Are there missing values, duplicates, bad formats or noise?
- Are images correctly labeled?
- Is text relevant and usable?
- Are documents current and relevant for RAG?
- Are privacy, licensing and access restrictions understood?

🧩 6. CHOOSE THE RIGHT AI APPROACH

🖼️ **DL / CV** – Image Classification or Object Detection when visual information is central.

📝 **NLP** – Text Classification when text must be assigned to predefined categories.

📖 **GenAI / RAG** – Grounded answers from relevant industry documents.

🧑 **AI Agent** – Multi-step reasoning/action, tool use or workflow orchestration – only when justified.

💡 GOOD PRACTICE

Use the simplest approach that solves the problem well. Complexity should be justified by business value, not technology trends.

7. DL / COMPUTER VISION

Image Classification

- Define classes and collect representative labeled images.
- Validate class balance, image quality, duplicates and labels.
- Prepare images with preprocessing/augmentation.
- Use transfer learning or a suitable DL architecture when appropriate.
- Evaluate metrics and inspect incorrect predictions.
- Save model and inference preprocessing.

Object Detection

- Define object classes and bounding-box annotations.
- Validate annotation quality and class distribution.
- Train/fine-tune a suitable detection model.
- Evaluate detection metrics and inspect misses/false detections.
- Test on realistic images/video before deployment.



REMEMBER

Classification answers "What is in this image?" Object Detection answers "What objects are present and where?"

8. NLP – TEXT CLASSIFICATION

- Define categories and labeling rules.
- Collect and validate representative text.
- Clean/normalize only where appropriate.
- Choose a suitable representation/model approach.
- Train, compare and evaluate.
- Inspect misclassified examples and improve.
- Save model and preprocessing/tokenization components.

9. GENAI FEATURE – RAG

Add one genuinely useful GenAI feature. A strong default is Retrieval-Augmented Generation (RAG) over relevant industry documents.

RAG WORKFLOW

User question → retrieve relevant document chunks → provide context to the LLM → generate a grounded answer → show source references where possible.

-  Collect relevant industry documents.
-  Clean and prepare documents.
-  Split into suitable chunks.
-  Create embeddings and store them in a vector store.
-  Retrieve relevant chunks for a query.
-  Send retrieved context to the LLM.
-  Generate an answer from the context.
-  Show document/source references where possible.

10. AI AGENT – IF JUSTIFIED

Use an Agent when the application needs AI to decide which tools/actions to use as part of a workflow.

AGENT WORKFLOW

User request → Agent understands goal → chooses tool(s) → retrieves/calculates/acts → checks result → responds.

-  Tools may include APIs, databases, search, calculators or application functions.
-  Define what the agent is allowed to do.
-  Apply authentication, authorization and safety controls.
-  Test incorrect requests, tool failures and unexpected outputs.
-  Log important agent actions for debugging and evaluation.

DO NOT FORCE IT

If a normal function, API call or deterministic workflow solves the problem, use that instead of an Agent.

11. EVALUATION

Component	Evaluation
 Image Classification	Accuracy / Precision / Recall / F1 + error analysis.
 Object Detection	Detection metrics + visual inspection.
 NLP	Precision / Recall / F1 + error analysis.
 RAG	Retrieval relevance + groundedness/faithfulness + usefulness.
 Agent	Task completion + tool selection/use + failure handling + safety.
 Application	Usability, latency, reliability and end-to-end success.

12. INTEGRATE INTO ONE APPLICATION

Do not create disconnected CV, NLP, RAG and Agent demos. Where justified, integrate the selected components into one coherent application.

- 🗑️ Clear user interface.
- 🔌 Connect model/API components cleanly.
- 📖 Connect RAG where relevant.
- 🏠 Connect Agent only where it adds workflow value.
- 🗄️ Use databases/APIs for persistent or live data.
- ⚠️ Handle invalid inputs and service failures.
- 📊 Clearly show predictions, extracted results or generated responses.

INTEGRATION RULE

Every component must have a clear job. If removing a component does not reduce application value, question why it is there.

13. DEPLOYMENT

Choose deployment based on architecture, component requirements, scalability, cost and security.

- 💻 Streamlit – simple interactive applications and demos.
- ⚡ FastAPI / Flask – APIs for models and services.
- 📦 Docker – package complex applications/services.
- ☁️ Cloud platforms – deploy APIs, applications, databases, model services or infrastructure.
- 🔒 Never commit API keys, credentials, private documents or sensitive data to GitHub.

DEPLOYMENT MINDSET

Do not force every AI component into one hosting method. Choose an architecture the application can actually support.

💰 14. BUSINESS IMPACT

- Can the intended user actually use it?
- Does it reduce time, cost, errors or manual work?
- Does it improve a decision or workflow?
- Is the AI output trustworthy enough for the intended use?
- What are the limitations and risks?



REMEMBER

A technically impressive AI demo is not automatically a useful business solution.

🔄 15. MONITORING & IMPROVEMENT

- 📊 Model performance when ground truth becomes available.
- 🗑️ Data drift and input quality.
- 📖 Document changes and RAG retrieval quality.
- 🧠 LLM behavior and cost.
- 🧑 Agent actions, failures and tool usage.
- 🔒 Security, access and sensitive-data handling.
- 🔧 Improve/retrain components when evidence shows a need.

INTERVIEW / PROJECT REVIEW QUESTIONS

- ? Why did you choose this industry and domain?
- ? What is the idea and who benefits?
- ? Why is AI useful for this idea?
- ? Where did the data/documents come from?
- ? What is the exact technical problem?
- ? Why did you choose DL, NLP, RAG or an Agent?
- ? Why not the alternatives?
- ? How did you validate your data?
- ? How did you evaluate each AI component?
- ? How do you know the RAG answer is grounded?
- ? What tools can the Agent use and why?
- ? What happens when a model, LLM, retrieval system or tool fails?
- ? How does the application create business value?
- ? What are the limitations, risks and improvements?

📁 FINAL PROJECT STRUCTURE

Example only – adapt it to the actual application and deployment approach.

```
AI-Project/  
├── 📁 data/  
│   ├── raw/  
│   ├── processed/  
│   └── external/  
├── 📁 notebooks/  
├── 📁 models/  
│   ├── ml/  
│   ├── cv/  
│   └── nlp/  
├── 📁 app/  
│   ├── backend/  
│   ├── frontend/  
│   ├── rag/  
│   └── agent/  
├── 📁 documents/  
├── 📁 reports/  
├── requirements.txt  
├── README.md  
└── .gitignore
```

⚠️ REMEMBER

Do not create empty folders to match this structure. Adapt it to the actual project, deployment approach and components used.